

Chapter

Debugging in Python 3

Interactive Inspection with pdb, Stack Navigation, and the Surrounding Code-Quality Toolchain

Table of Contents

Table of Contents	2
Abstract	4
Part I — Theoretical Foundations	5
1. The Role of Interactive Debugging	5
2. pdb: Python's Built-In Debugger	5
3. Starting a Debugger Session	5
4. The pdb Command Model	6
5. Inspecting Program State	6
5.1 Printing Variables with p	6
5.2 Pretty-Printing Complex Objects with pp	6
5.3 Evaluating Arbitrary Expressions	7
6. Controlling Execution Flow	7
6.1 next (n): Stepping Over	7
6.2 step (s): Stepping Into	7
6.3 continue (c): Resuming Until the Next Breakpoint	7
6.4 quit (q): Terminating the Session	7
7. Breakpoints	8
8. Navigating the Call Stack	8
9. Reading a Stack Trace	9
10. Displaying Source Code	9
11. Adjacent Tools in the Debugging and Code-Quality Workflow	9
11.1 PyLint: Static Code Analysis	9
11.2 ipdb: An Enhanced pdb	10
11.3 Black: Automated Code Formatting	10
11.4 isort: Automated Import Sorting	10
11.5 mypy: Static Type Checking	10
Part II — Practical Implementation and Annotated Code	12
12. Starting and Using pdb	12
12.1 The Minimal set_trace() Call	12
12.2 Pausing Inside a Function	12
12.3 Using breakpoint() Instead of pdb.set_trace()	12
12.4 Launching the Script Normally	13
13. Inspecting State at the (Pdb) Prompt	13

13.1 General and Command-Specific Help	13
13.2 Printing Variable Values	13
13.3 Pretty-Printing a Complex Object	14
13.4 Evaluating Derived Expressions	14
14. Controlling Execution from the Prompt	14
14.1 Stepping Over with next	14
14.2 Resuming Until the Next Breakpoint	15
14.3 Exiting the Debugger	15
15. Working with Breakpoints	15
15.1 Setting a Breakpoint by Line Number	15
15.2 Listing Active Breakpoints	15
15.3 Example Breakpoint Listing Output	16
15.4 A Conditional Breakpoint	16
15.5 Clearing Breakpoints	16
16. Stepping Into Function Calls	16
16.1 The step Command	16
16.2 A Concrete step Scenario	17
17. Navigating the Call Stack	17
17.1 Moving Up to the Caller's Frame	17
17.2 Moving Back Down to a Previously Viewed Frame	17
18. Reading the Full Call Stack	18
18.1 Displaying a Stack Trace	18
19. Displaying Source Code from Within the Debugger	18
19.1 Listing Nearby Lines	18
19.2 Listing the Entire Current Function	18
20. The Surrounding Toolchain in Practice	19
20.1 Running PyLint	19
20.2 Installing ipdb	19
20.3 Formatting with Black	19
20.4 Sorting Imports with isort	20
Summary	20

Abstract

Debugging is the systematic process of locating the point at which a program's actual behavior diverges from its intended behavior, and interactive debugging — pausing execution to inspect live program state — is the most direct instrument available for that purpose, complementing the static, print-statement-based, and profiling-based techniques used elsewhere in this book. Python's standard library ships `pdb`, a command-line interactive debugger supporting breakpoints, stepping, expression evaluation, and stack navigation, accessible with minimal code changes via `pdb.set_trace()` or, from Python 3.7 onward, the `breakpoint()` built-in. This chapter develops the theoretical model underlying `pdb` — the notion of a paused execution frame, a navigable call stack, and a command vocabulary for advancing or inspecting that paused state — before treating each command individually, and closes with the adjacent static-analysis and formatting tools (PyLint, `ipdb`, Black, `isort`, `mypy`) that address code correctness and consistency at a different point in the development lifecycle than interactive debugging does. As in the preceding chapters, Part I develops the underlying theory and Part II operationalizes it through annotated, cell-level code listings.

Part I — Theoretical Foundations

1. The Role of Interactive Debugging

Interactive debugging addresses a specific limitation of static reading and print-statement instrumentation: neither reveals the actual runtime value of every variable at an arbitrary point in execution without the programmer having anticipated, in advance, exactly which values would be worth inspecting. An interactive debugger instead suspends the running program at a chosen point and exposes a live, queryable view of its state — local and global variables, the call stack, and the ability to evaluate arbitrary expressions in that paused context — converting debugging from a process of guessing which print statements to add and re-running the program repeatedly, into a process of directly interrogating the program's actual state at the moment something goes wrong or looks suspicious.

2. pdb: Python's Built-In Debugger

pdb (Python Debugger) is a command-line, source-level debugger included in the Python standard library, requiring no external installation and minimal code modification to use. It operates by suspending normal execution at a designated point and replacing the running program's control flow with an interactive (Pdb) prompt, at which the developer issues commands that either inspect the current state or advance execution by a controlled increment (one line, one function call, until the next breakpoint, and so on). Because pdb is a standard-library module, it is universally available in any Python 3 environment without dependency management, making it the default first tool for interactive debugging even where richer third-party alternatives (such as ipdb, §11.2) exist.

3. Starting a Debugger Session

A pdb session is initiated by inserting a call to `pdb.set_trace()` at the exact line where inspection should begin; when the interpreter reaches that call during normal execution, it suspends the program immediately afterward, prints the current source location, and presents the (Pdb) prompt in the terminal in place of further automatic execution. From Python 3.7 onward, the built-in `breakpoint()` function is the recommended equivalent: it is functionally identical to `pdb.set_trace()` but requires no explicit import statement, since `breakpoint()` is itself a built-in that internally delegates to pdb (or to whatever debugger is configured via the `PYTHONBREAKPOINT` environment variable), making it both cleaner to write and more flexible to reconfigure without touching the calling code.

Once the (Pdb) prompt appears, the developer has full access to the current scope: every local variable visible at that line, every global, and the ability to evaluate arbitrary Python expressions against that live state, as detailed in §§5–6.

4. The pdb Command Model

Every pdb command is entered at the (Pdb) prompt and falls into one of two broad categories: inspection commands, which query the current state without advancing execution (p, pp, l, ll, w, help), and execution-control commands, which advance the paused program by some defined increment (n, s, c, q) or modify where future pauses will occur (break, clear). Nearly every command has a short, one-letter alias (e.g., n for next, c for continue), reflecting that pdb commands are typically typed many times in rapid succession during an active debugging session, where brevity meaningfully reduces friction. The help command (or its alias h) is itself part of this vocabulary and can be invoked either generally, to list all available commands, or with a specific command name as an argument, to display that command's documentation — making the debugger partially self-documenting during use, without requiring the developer to consult external reference material mid-session.

5. Inspecting Program State

5.1 *Printing Variables with p*

The p command evaluates and prints the value of a given expression — typically a single variable name, but in general any valid Python expression — within the scope of the currently paused frame, exposing both local variables specific to the current function and any accessible global variables. Because p evaluates a full expression rather than merely echoing a variable's stored representation, it can also be used to compute and display derived values on the fly (§5.3) without needing to insert a corresponding statement into the actual source code.

5.2 *Pretty-Printing Complex Objects with pp*

pp behaves identically to p but formats its output using Python's pprint module rather than the default repr(), producing line-wrapped, indented output for nested or otherwise visually dense structures such as dictionaries, lists of dictionaries, or deeply nested JSON-derived objects. This distinction matters specifically because p's default formatting renders such structures as a single dense, hard-to-scan line, whereas pp's formatting preserves the structure's shape visually — making pp the preferred choice specifically when inspecting API responses, configuration objects, or any other naturally nested data encountered while debugging.

5.3 Evaluating Arbitrary Expressions

Because `p` (and `pp`) evaluate any valid Python expression, not merely bare variable names, the debugger can be used to test hypotheses about program behavior interactively — evaluating a transformed or derived value (`p quantity * 2`), calling a function on a live variable (`p len(investment)`), or otherwise probing 'what would this expression evaluate to right now' — without modifying the running program's source or restarting it. This capability is what elevates `pdb` from a passive state-viewer into an interactive investigation tool: hypotheses about a bug's cause can be tested directly against the program's actual live state in the moment, rather than requiring a source-code change and a full re-run for each hypothesis.

6. Controlling Execution Flow

6.1 next (n): Stepping Over

`next` advances execution by exactly one source line in the current frame, executing any function calls on that line to completion without pausing inside them — analogous to 'Step Over' in graphical IDE debuggers. This is the default choice when the current line's purpose is understood and the developer wishes to advance past it (including past any function calls it makes) without inspecting that called function's internals.

6.2 step (s): Stepping Into

`step` also advances execution by one statement, but — in contrast to `next` — if that statement is a function call, `step` pauses execution at the first line inside the called function rather than running it to completion, analogous to 'Step Into' in graphical IDE debuggers. `step` is the appropriate choice precisely when the function being called is itself suspected of containing the bug, or its internal behavior is otherwise the subject of investigation, since it extends the paused-state inspection capability (§5) into that function's own local scope.

6.3 continue (c): Resuming Until the Next Breakpoint

`continue` resumes normal, unpaused execution and runs until either the program terminates or another active breakpoint is reached, whichever occurs first. This is the efficient choice for skipping over large stretches of code already known to behave correctly, reserving the cost of step-by-step inspection (`next/step`) specifically for the region around a suspected fault, located via breakpoint placement (§7) rather than via exhaustive line-by-line stepping from the very start of the program.

6.4 quit (q): Terminating the Session

quit (aliased exit) immediately ends both the debugging session and the program being debugged, rather than allowing it to continue to completion. This is the appropriate command once the investigation is complete — whether because the cause has been identified or because the developer intends to modify the source and re-run from scratch — and should be distinguished clearly from continue, which resumes the program rather than terminating it.

7. Breakpoints

A breakpoint is a designated source location at which pdb will automatically pause execution, established dynamically during an active session via the break command (taking a line number or function name as its argument), in contrast to `pdb.set_trace()/breakpoint()`, which require a code modification to designate a pause point in advance. Because breakpoints can be set on the fly, at any point during a session, after an initial pause has already revealed information motivating further investigation elsewhere in the program, they offer a flexibility that purely code-embedded pause points do not.

- break with a line-number or function-name argument creates a new breakpoint at that location.
- break with no arguments lists every currently active breakpoint, along with its number, type, enabled/disabled status, and source location.
- Conditional breakpoints accept an additional boolean expression argument and pause execution only when that expression evaluates true on the relevant line — reducing the noise of pausing on every iteration of a loop when only a specific case (e.g., a specific value of a loop variable) is of interest.
- clear removes breakpoints, either a specific one by number or all of them at once, keeping a long debugging session's set of active pause points manageable.

8. Navigating the Call Stack

When the currently paused frame is nested inside one or more calling functions — the ordinary case once a program of any real depth is running — pdb maintains a navigable stack of those frames, allowing the developer to move between them without altering the program's actual execution state. The up command moves the debugger's focus one frame higher (toward the caller, analogous conceptually to 'stepping out' of the current function to inspect its caller's context), while down moves focus one frame lower (back toward the callee, typically used to return to a frame previously left via up).

Critically, navigating with up/down changes only which frame's variables `p`, `pp`, and `l` operate against — it does not execute any code or alter program state, making stack navigation a purely inspective, side-effect-free operation distinct from the execution-control commands in §6.

9. Reading a Stack Trace

The `where` command (aliased `bt`, for 'backtrace') displays the full current call stack — every active frame, from the program's outermost entry point down to the currently executing line — which is essential for understanding how execution arrived at its current point, particularly when diagnosing an exception whose ultimate cause lies several calls removed from where it was actually raised. By convention, the oldest (outermost) frame is listed first and the newest (innermost, currently executing) frame last; the current frame is marked with a `>` and the specific line about to execute within it is marked with `->`, giving an at-a-glance view of both the call history and the precise point of current execution within that history.

10. Displaying Source Code

Because a debugging session is conducted from the terminal rather than within a source-code editor, `pdb` provides commands to display source code directly, avoiding the need to repeatedly switch to a separate editor window to see the code surrounding the paused execution point. `list` (aliased `l`) displays a small window of source lines around the current execution point, or an explicitly specified range; `longlist` (aliased `ll`) instead displays the entirety of the current function, providing fuller context than `list`'s fixed-size window, which is particularly useful for longer functions where the immediate few lines of surrounding context may not be sufficient to understand the current execution point's role within the function as a whole.

11. Adjacent Tools in the Debugging and Code-Quality Workflow

Interactive debugging with `pdb` addresses runtime behavior at a specific moment in a specific execution; it does not, by itself, catch defects that can be identified without ever running the program, nor does it enforce stylistic or structural consistency across a codebase. A complementary toolchain — static analysis, enhanced debugger front-ends, and automated formatting — addresses these adjacent concerns and is typically used alongside, not instead of, `pdb`.

11.1 PyLint: Static Code Analysis

PyLint analyzes Python source code without executing it, detecting a broad class of issues — undeclared variables, unused imports, style violations, certain categories of likely bugs — purely from the program's static structure. Because it requires no execution, PyLint can surface issues before a program is ever run, including in code paths that a given test run or manual session would not happen to exercise, making it a useful complement to runtime-only tools such as pdb.

11.2 ipdb: An Enhanced pdb

ipdb is a third-party debugger built directly on top of pdb, providing the identical command vocabulary described in §§4–10 while adding usability improvements such as syntax highlighting and integration with IPython's tab-completion and introspection features. Because it is API-compatible with pdb, knowledge of pdb's commands transfers directly to ipdb; it is best understood as a higher-fidelity front end over the same underlying debugging model rather than a distinct tool requiring separate study.

11.3 Black: Automated Code Formatting

Black reformats Python source code into a single, consistent, deterministic style, eliminating the need for a team to debate or manually enforce formatting conventions (line length, quote style, spacing) and ensuring that formatting differences never appear as noise in code review or version-control diffs. Because Black's formatting is deterministic given a fixed input, running it repeatedly on already-formatted code produces no further changes, making it safe to run routinely (e.g., as part of a pre-commit hook) without risk of unexpected, repeated reformatting.

11.4 isort: Automated Import Sorting

isort automatically reorders and groups a module's import statements according to a consistent convention (standard library, third-party, local imports, each alphabetized within its group), addressing a narrower but frequently bikeshedded concern than Black's general formatting. isort is commonly configured to run alongside Black specifically so that import organization and general code formatting are both handled automatically and consistently, rather than one being automated and the other left to manual discipline.

11.5 mypy: Static Type Checking

mypy analyzes a codebase's type hints (where present) and reports inconsistencies — passing a string where an int is expected, calling a method that the declared type does not define — without executing the code, in the same static-analysis spirit as PyLint but specifically focused on type correctness rather than general code-quality concerns.

Because Python's type hints are optional and not enforced at runtime by the interpreter itself, mypy is the mechanism by which an explicitly typed Python codebase gains the bug-catching benefit of static type checking that the interpreter alone does not provide, catching a category of error (type mismatches) that neither pdb's runtime inspection nor PyLint's general static analysis specifically targets.

Part II — Practical Implementation and Annotated Code

Each listing below corresponds to a concept introduced in Part I and is treated as an independent notebook cell or terminal interaction. Explanations are given at the what/why level for the cell as a whole, consistent with the convention established in the companion chapters. Where a listing represents commands typed at the interactive (Pdb) prompt rather than a runnable script cell, this is noted explicitly.

12. Starting and Using pdb

12.1 The Minimal `set_trace()` Call

```
import pdb
pdb.set_trace()
print("Debugging started")
```

Cell explanation: Importing `pdb` and immediately calling `pdb.set_trace()` suspends the program at that exact point, before the subsequent `print` statement executes; the terminal then presents the (Pdb) prompt instead of continuing automatically. The purpose of this minimal cell is to isolate the single line responsible for triggering a debugging pause, independent of any surrounding application logic, establishing the baseline mechanism that every other `pdb` example in this chapter builds on.

12.2 Pausing Inside a Function

```
def calculate():
    x = 10
    y = 20
    import pdb
    pdb.set_trace()
    result = x + y
    return result

calculate()
```

Cell explanation: Here `set_trace()` is placed inside `calculate()`, after `x` and `y` have been assigned but before `result` is computed — meaning that once paused, both `x` (10) and `y` (20) are already valid, inspectable local variables (via `p x` or `p y`, §13.1), while `result` does not yet exist. This cell demonstrates that `set_trace()`'s pause point is exact and line-specific: which variables are available for inspection depends precisely on which statements have already executed relative to where the call is placed, not on the function as a whole.

12.3 Using `breakpoint()` Instead of `pdb.set_trace()`

```
def calculate():
```

```
x = 10
y = 20
breakpoint()
return x + y
```

Cell explanation: `breakpoint()` achieves the identical pausing effect as §12.2's `import pdb; pdb.set_trace()` pair, but as a single built-in call requiring no explicit import statement. This cell is functionally equivalent to §12.2 (modulo the inlined `return` expression) and exists specifically to contrast the two forms, reinforcing Part I §3's recommendation that `breakpoint()` — being shorter, requiring no import, and being reconfigurable via `PYTHONBREAKPOINT` without touching this line — is the preferred form in any Python 3.7+ codebase.

12.4 Launching the Script Normally

```
python app.py
```

Cell explanation: This is a terminal command, not a Python cell: it runs `app.py` exactly as any ordinary script would be run, with no special debugger flag required, since the `breakpoint()/set_trace()` call embedded in the script's own source (as in §12.1–§12.3) is what triggers the pause — the debugger activates from inside the running program itself rather than from any external invocation option. When the interpreter reaches that embedded call, the terminal session running this exact command transitions into the interactive (Pdb) prompt, illustrating Part I §3's description of how a session actually begins from the developer's point of view.

13. Inspecting State at the (Pdb) Prompt

13.1 General and Command-Specific Help

```
(Pdb) help
(Pdb) help next
```

Cell explanation: These are commands typed at the (Pdb) prompt, not Python source. `help` with no argument lists every available pdb command; `help next` instead prints the specific documentation for the next command alone. The purpose of pairing these two forms is to show that `help` itself follows the general/specific pattern common across pdb's command vocabulary — useful both for a first-time user discovering what commands exist, and for an experienced user double-checking one specific command's exact behavior.

13.2 Printing Variable Values

```
(Pdb) p name
(Pdb) p quantity
```

Cell explanation: Each line independently evaluates and prints the named variable (name, then quantity) as it currently exists in the paused frame's scope. The purpose of showing two separate `p` invocations is to emphasize that `p` is a general-purpose command applied repeatedly, once per variable of interest, rather than a single command that dumps all local state at once — the developer chooses, on a per-command basis, exactly which piece of state to inspect at any given moment.

13.3 Pretty-Printing a Complex Object

```
(Pdb) pp investment
```

Cell explanation: Where `p investment` (§13.2's pattern, applied to a presumably nested object) would print `investment's repr()` on a single, potentially unreadable line, `pp investment` instead formats it with line breaks and indentation reflecting its actual structure. The purpose of this cell, read against §13.2, is to make concrete exactly when `pp`'s extra formatting effort is worthwhile — specifically for an object such as `investment` that is presumed here to be a dictionary or other nested structure, where `pp`'s readability benefit over `p` is most pronounced.

13.4 Evaluating Derived Expressions

```
(Pdb) p quantity * 2  
(Pdb) p len(investment)
```

Cell explanation: Both lines pass a full expression, not a bare variable name, to `p`: the first computes and prints double `quantity`'s current value; the second computes and prints the length of `investment`. Neither expression needs to exist anywhere in the actual source code — they are constructed entirely at the prompt, on the fly — directly demonstrating Part I §5.3's claim that `pdb` enables testing arbitrary hypotheses about the program's live state without any corresponding source-code change.

14. Controlling Execution from the Prompt

14.1 Stepping Over with `next`

```
(Pdb) next  
(Pdb) n
```

Cell explanation: Both lines issue the identical command — `next`, then its one-letter alias `n` — each advancing execution by exactly one line in the current frame, running any function call on that line to completion without pausing inside it. They are shown together specifically to establish that the long form and the alias are fully interchangeable, which is true throughout `pdb`'s command set (§4), so that subsequent examples in this chapter can use either form without loss of clarity.

14.2 Resuming Until the Next Breakpoint

```
(Pdb) continue  
(Pdb) c
```

Cell explanation: `continue` (and its alias `c`) resumes ordinary, unpaused execution, which will proceed until the program ends or another active breakpoint (§15) is reached. Issued from a session that may have multiple breakpoints set, this command's purpose is to skip rapidly over a stretch of code already considered uninteresting, deferring the next pause to wherever the next breakpoint actually lies, rather than continuing to step line by line through code already understood to be correct.

14.3 Exiting the Debugger

```
(Pdb) quit  
(Pdb) q
```

Cell explanation: `quit` (and its alias `q`) ends the debugging session and terminates the program being debugged immediately, rather than letting it run to completion. This is the command issued once the developer is finished investigating — whether because the bug's cause has been found or because they intend to edit the source and restart — and is presented last among the execution-control commands specifically because it is the terminal action that ends the session, in contrast to `next/continue`, which keep the session active.

15. Working with Breakpoints

15.1 Setting a Breakpoint by Line Number

```
(Pdb) break 30
```

Cell explanation: This command instructs `pdb` to pause execution automatically whenever line 30 of the current file is reached, without requiring a `pdb.set_trace()` call to have been embedded in the source at that line. The purpose is to demonstrate the dynamic, session-time alternative to code-embedded pause points described in Part I §7 — a breakpoint can be added on the fly at any point during an already-active session, targeting a location that interests the developer only after the session has already begun and revealed something worth investigating further elsewhere.

15.2 Listing Active Breakpoints

```
(Pdb) break
```

Cell explanation: Calling `break` with no arguments switches its behavior from 'create a breakpoint' to 'list existing breakpoints', printing a table (illustrated in §15.3) of every breakpoint currently set in the session. This dual behavior — same command name,

different effect depending on whether an argument is supplied — is a recurring pattern worth noting explicitly, since it means `break`'s meaning is determined entirely by the presence or absence of its argument.

15.3 Example Breakpoint Listing Output

```
Num Type      Disp Enb Where
1 breakpoint keep yes at app.py:30
```

Cell explanation: This is the terminal output produced by §15.2's bare `break` command, not a command itself. Each column reports one breakpoint's identifying number (Num), its type, its disposition (Disp — keep, meaning it remains active after being hit, as opposed to being removed after a single trigger), whether it is currently enabled (Enb), and its source location (Where). The purpose of including this output explicitly is to make the otherwise-abstract 'breakpoints can be listed' claim in Part I §7 concrete by showing exactly what that listing looks like and what each field means.

15.4 A Conditional Breakpoint

```
(Pdb) break 30 quantity > 100
```

Cell explanation: Appending `quantity > 100` after the line number restricts this breakpoint so that execution pauses at line 30 only when that boolean expression evaluates to true at that point, rather than on every single pass through line 30. This is the direct realization of Part I §7's conditional-breakpoint description, and is specifically valuable inside a loop where line 30 might execute many times but only a particular case (here, when quantity exceeds 100) is actually of interest.

15.5 Clearing Breakpoints

```
(Pdb) clear
(Pdb) clear 1
```

Cell explanation: `clear` with no arguments removes every breakpoint currently set in the session (typically prompting for confirmation); `clear 1` instead removes only the specific breakpoint numbered 1, as identified in a prior break listing (§15.2–§15.3). The purpose of showing both forms together is to demonstrate that breakpoint management, like creation, supports both a broad, session-wide action and a precise, single-breakpoint action, letting the developer keep a long session's set of active pause points deliberately curated rather than accumulating unmanaged clutter.

16. Stepping Into Function Calls

16.1 The `step` Command

```
(Pdb) step
```

```
(Pdb) s
```

Cell explanation: `step` (and its alias `s`) advances execution by one statement, but — unlike `next` (§14.1) — if that statement invokes a function, `step` pauses at the very first line inside that called function rather than running it to completion. This command pair is shown to directly contrast with §14.1's `next`, reinforcing Part I §6.2's distinction: the two commands differ in exactly one respect — whether a function call encountered along the way is stepped into or stepped over — despite otherwise both advancing execution by 'one statement.'

16.2 A Concrete step Scenario

```
def get_current_price(coin):  
    print("Getting current price...")  
    return 10000  
  
price = get_current_price("bitcoin")
```

Cell explanation: If a debugger session were paused on the `price = get_current_price("bitcoin")` line and the developer issued `step` rather than `next`, execution would pause immediately inside `get_current_price`, before its own `print` statement has run — allowing inspection of the `coin` parameter's value ("bitcoin") from inside the called function's own scope. This cell exists specifically to give §16.1's abstract `step/next` distinction a concrete function to apply it to, making clear exactly what 'pausing inside the called function' means in terms of an actual, nameable line of code.

17. Navigating the Call Stack

17.1 Moving Up to the Caller's Frame

```
(Pdb) up  
(Pdb) u
```

Cell explanation: `up` (and its alias `u`) shifts the debugger's current focus one frame higher in the call stack — toward the function that called the currently-focused function — without executing any further code or altering program state. After issuing this command, subsequent `p/pp/l` commands operate against the caller's local variables instead of the previously-focused frame's, which is the mechanism described in Part I §8 for inspecting how and from where the current point of execution was actually reached.

17.2 Moving Back Down to a Previously Viewed Frame

```
(Pdb) down
```

```
(Pdb) d
```

Cell explanation: down (and its alias d) reverses an up: it shifts focus one frame lower, back toward the callee, and is typically used after one or more up commands to return toward the originally-paused frame once the caller's context has been sufficiently inspected. Presenting up and down as a pair reflects that they are inverse operations operating on the same underlying stack-navigation mechanism — neither alters execution; both only change which frame's state is currently visible to inspection commands.

18. Reading the Full Call Stack

18.1 Displaying a Stack Trace

```
(Pdb) where  
(Pdb) w
```

Cell explanation: where (aliased w, and also bt for 'backtrace') prints every active frame in the current call stack at once, from the outermost caller down to the currently executing line, rather than requiring repeated up/down navigation to view each frame individually. This single command gives the full execution-path overview described in Part I §9, with the current frame marked by > and the current line within it marked by ->, making it the natural first command to run when first investigating an unfamiliar or unexpected pause point — particularly one resulting from an exception raised several calls below where it ultimately propagated.

19. Displaying Source Code from Within the Debugger

19.1 Listing Nearby Lines

```
(Pdb) list  
(Pdb) l
```

Cell explanation: list (and its alias l) prints a small window of source lines surrounding the current execution point, giving immediate local context without switching to a separate editor window. This is the default choice for a quick 'where exactly am I in this function' check, in contrast to longlist (§19.2), which is reserved for cases where the immediate few lines of context are not sufficient.

19.2 Listing the Entire Current Function

```
(Pdb) longlist  
(Pdb) ll
```

Cell explanation: `longlist` (and its alias `ll`) prints the complete body of the function containing the current execution point, with that point highlighted, rather than `list`'s fixed-size local window. This is the appropriate choice specifically when understanding the current line's role requires seeing the function as a whole — its parameters, its overall control flow, and where in that flow the current line sits — which a short, fixed window of nearby lines may not adequately convey for a longer function.

20. The Surrounding Toolchain in Practice

20.1 Running PyLint

```
pylint app.py
```

Cell explanation: This terminal command runs PyLint's static analysis against `app.py` without executing any of its code, printing a report of detected issues (style violations, unused variables, likely bugs) directly to the terminal. The purpose of including this alongside the interactive `pdb` commands above is to make explicit, by direct juxtaposition, that PyLint addresses a different point in the workflow than `pdb` does: issues here are found before the program ever runs, whereas every `pdb` command in this chapter operates on a program that is actively executing.

20.2 Installing ipdb

```
pip install ipdb
```

Cell explanation: `ipdb` is a third-party package and therefore requires installation via `pip`, unlike `pdb`, which ships with the standard library and needs no such step. The purpose of showing this single installation command is to mark the precise boundary between the two tools' availability: every `pdb` command demonstrated earlier in this chapter works identically once `ipdb` is installed and substituted in (per Part I §11.2), with this command being the only additional step required to gain `ipdb`'s added usability features.

20.3 Formatting with Black

```
black app.py
```

Cell explanation: Running this command reformats `app.py` in place according to Black's fixed, opinionated style rules, with no configuration required for the common case. Its purpose here is to represent the automated-formatting end of the toolchain spectrum introduced in §11 — a tool whose effect is applied to the entire file at once and concerns presentation rather than runtime behavior, in direct contrast to `pdb`'s line-by-line, runtime-focused operation.

20.4 Sorting Imports with isort

```
isort app.py
```

Cell explanation: This command reorganizes `app.py`'s import statements into `isort`'s standard grouping and ordering convention, addressing the narrower, import-specific subset of formatting concerns that `Black` does not itself enforce — included here specifically to show that `isort` and `Black` are typically run as a complementary pair (per Part I §11.4) rather than as alternatives to one another, each automating a distinct facet of code presentation.

Summary

Interactive debugging with `pdb` operates on a single underlying model: execution can be paused at a chosen point (via `pdb.set_trace()`, `breakpoint()`, or a dynamically-set break), and once paused, the program's live state — variables in the current frame (`p`, `pp`), the full call stack (`where`, `up`, `down`), and the surrounding source code (`list`, `longlist`) — becomes directly inspectable, while a small set of execution-control commands (`next`, `step`, `continue`, `quit`) determine how and whether execution subsequently resumes. Every command introduced in this chapter is an instance of one of these two roles — inspecting paused state, or controlling the resumption of execution — and that two-part model is sufficient to navigate essentially any interactive debugging session, whether conducted with `pdb` directly or with a richer front-end such as `ipdb`. This runtime-focused toolset is deliberately complemented, not replaced, by the static tools introduced in §11 — `PyLint` and `mypy` for catching issues before execution, `Black` and `isort` for consistent presentation — each addressing a concern that pausing a running program cannot, by itself, address.